Rental Chatbot and Agent Triage

Anslean Albert Jeyaras

***Abstract*—This report presents a prototype for handling rental enquiries with large language models (LLMs) while keeping a human in control of every decision. A chatbot answers prospective tenants' questions and extracts their details into a structured enquiry. A separate triage agent then recommends whether the enquiry is genuine, missing information, spam or a duplicate, and a human reviewer approves or overrides the recommendation. Both agents use only read-only tools. Personal data is redacted before triage, and every recommendation must pass schema, identity and evidence checks before it can reach the reviewer. Over six runs on 100 fictional enquiries, triage achieved a mean accuracy of 96.7%. All 8 failed runs were rejected by the guards and never reached the review queue. Extraction was correct for 98% of 150 transcript runs, with no invented values. The results show that guards and human review keep model errors from becoming decisions, although the evaluation uses synthetic data and does not predict performance on real enquiries.**

***Index Terms*—large language models, human-in-the-loop, LLM agents, tool use, privacy, text classification, information extraction**

I. Introduction

Large language models are increasingly deployed as agents that take actions on a user's behalf rather than only generating text, and real estate lead management is a natural candidate for this shift: enquiries that go unanswered for even a short time lose much of their conversion value, so listings benefit from a fast, consistent first response [1]. This project implements a pipeline in which agents handle that first response end to end: a chatbot gathers and extracts structured information from an enquiry, and a separate triage agent classifies it as genuine, needs more information, spam, or a duplicate of an existing enquiry. Because this pipeline makes decisions that affect real contacts and real leads, the agent's authority is deliberately constrained, with every recommendation subject to human review and audit. Evaluated across repeated runs on a fixed set of labeled enquiries, the triage agent correctly classifies the great majority of cases. The rest of this paper describes the design decisions behind the pipeline, the data used to build and evaluate it, the system architecture, and the evaluation results.

II. Data Model

The system stores two kinds of data. *Reference data* (locations, buildings and listings) is held in `data/` and only read at runtime. *Runtime records* (enquiries, recommendations and reviews) are written to `output/` by application code, and a per-run event log is appended to `audit/`. All entities are defined as Pydantic models, so every record is validated when it is created and again when it is loaded from disk.

A. Data Description

The seed data comprises fictional listings and buildings in Montreal and Copenhagen. The two locations share the same dataset structure; using two cities was intended to increase variability in the test data and to check whether the chatbot responds correctly in multiple languages (English, French and Danish).

B. Reference Data

Reference data forms a simple hierarchy: each *Listing* belongs to a *Building* (`building_id`), and each building belongs to a *Location* (`location_id`). The chatbot and the triage agent access this data only through read-only lookup and search tools.

C. Runtime Records

Table I summarises the runtime entities, and Fig. 1 shows how they are linked.

TABLE I
Runtime entities

Entity	Key	Purpose
Enquiry	`enquiry_id`	Details extracted by the chatbot
Recommendation	`run_id`	Triage agent's proposed label and evidence
Review	`enquiry_id`, `run_id`	Human decision on one recommendation
Event	`run_id`	Append-only log of one triage run

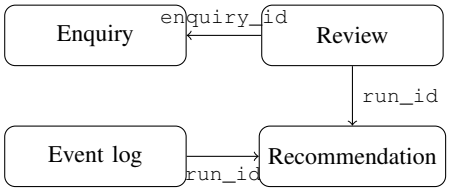


Fig. 1. Links between runtime records. One enquiry may be triaged in several runs; each run has its own recommendation, review and event log.

Enquiry. An enquiry holds the details the chatbot extracts from a conversation: name, email, phone, budget and currency, move-in date, listing and a free-text message. Every field is optional, because a conversation may end before all details are given; whether an enquiry is complete is therefore decided during triage, not by the schema.

Presence flags. Before triage, the application converts an enquiry into a redacted view. Contact details are replaced by

two booleans: `has_name` (a non-blank name is present) and `has_contact` (a non-blank email *or* phone is present). The flags are computed from the structured fields only; details mentioned only in the message do not count.

Run identifier. Each triage run is identified by a `run_id` (UUID) generated by the application, not by the model. It links the recommendation, the review and the event log of that run.

Recommendation. This is the schema the triage agent must fill. It contains a `label` (`genuine`, `needs_info`, `spam` or `duplicate`), a `duplicated_enquiry_id` (required if and only if the label is `duplicate`), a short `reason`, and between one and five *evidence* items. Each evidence item names its `source` (the message or a tool result), a verbatim `quote` of at least eight characters, and an explanation of how the quote `supports` the label.

Review. A review is created in the `pending` state when a recommendation is saved, and moves once to either `approved` or `overridden`. Model validators enforce its consistency: a decided review must record a final label, an override must include a message explaining it, an enquiry cannot be marked as a duplicate of itself, and the review must match the recommendation it refers to.

Event log. Each run appends events (run start, model calls, tool calls and results, guard checks, the recommendation and run end) to a JSONL file. Model-call events record token usage, which is used for cost estimation, and tool-result events are used to check evidence quotes.

III. Decisions

The key design decisions and their trade-offs are summarised below.

- **Two separate agents.** The system has two agent processes: a chatbot that answers prospective tenants' questions and collects their details, and a triage agent that classifies each submitted enquiry as `genuine`, `needs_information`, `spam` or `duplicate`. Separating them keeps each prompt and tool set narrow, and allows each to be evaluated on its own (see below).
- **A human makes the final decision.** The triage agent only recommends a label; a reviewer approves or overrides it. Decided reviews are write-once. Keeping a person in the loop means a misclassification, such as a genuine tenant marked as spam, can be caught before it has any effect. Trade-off: the tool assumes a single reviewer, so concurrent reviews are not handled.
- **JSON files instead of a database.** Seed inputs are stored in `data/` and runtime state in `output/`. For a small prototype, this avoids database setup and keeps every record human-readable. Each submission produces three files (the enquiry, the recommendation and a pending review), each written atomically. The review is written last, so an interrupted save can never place an incomplete record in the review queue. Trade-off: the three writes are not a single transaction, so an interruption may leave an orphaned enquiry or recommendation.
- **Read-only tools for the model.** The model can only look up listings, buildings, locations and existing enquiries. All saving is done by application code outside the tool loop, so the model can never modify stored data and no write-permission flag is needed. Trade-off: any new action that needs to write must be added to application code rather than exposed as a tool.
- **Personal data is filtered before triage.** Duplicate matching runs locally against the original contact details. The model receives only redacted text and presence flags, and tool results and logged tool arguments are filtered as well. This decision is to comply with Canadian and Quebec law [2] [3]. The human reviewer still sees the original details. Trade-off: redaction is regex-based, so it misses partial names and wrongly redacts some numeric lists.
- **A deliberately simple completeness rule.** Once spam and duplicates are ruled out, an enquiry is `genuine` only if it has a name and either an email or a phone number; budget, listing and move-in date are optional. The rule uses structured-field presence flags rather than re-reading the conversation, since extracting details is the chatbot's job. Trade-off: an enquiry with contact details but no indication of what the tenant wants still counts as complete.
- **Recommendations are validated against evidence.** The model fills a Pydantic schema, the reply is validated locally, and every quoted piece of evidence is checked against the redacted message or the recorded tool result. Trade-off: a correct quote does not prove the reasoning built on it is correct.
- **Triage and extraction are evaluated separately.** Triage is tested on 100 fictional enquiries; chatbot extraction on 25 fixed transcripts, scored on the final extracted fields and refusals only, with no per-turn checks. Separating the two makes it clear which component caused an error. Answer keys are never included in model input, and failed or truncated outputs are counted as failures rather than discarded. Trade-off: the transcripts are fixed, so the evaluation does not test how the chatbot handles a live, changing conversation.

IV. Architecture

Fig. 2 shows the system. A single command (`main.py`) runs three stages in sequence: a *chatbot* that collects a rental enquiry, a *triage agent* that recommends a label for it, and a *human review* step where a reviewer makes the final decision. Each stage is a separate module and can also be run on its own: the evaluation reuses the triage stage, and the review stage can be started later with `python -m core.manual_review`.

A. Chatbot

The user converses with an LLM-based chatbot that answers questions about available rentals. When the user ends the conversation, a single extraction call converts the transcript into a structured *Enquiry*, which is saved to `output/` immediately. Saving before triage ensures that the tenant's details are kept even if triage later fails. The model used for this project is Claude's *Haiku 5.5* using its SDK [4].

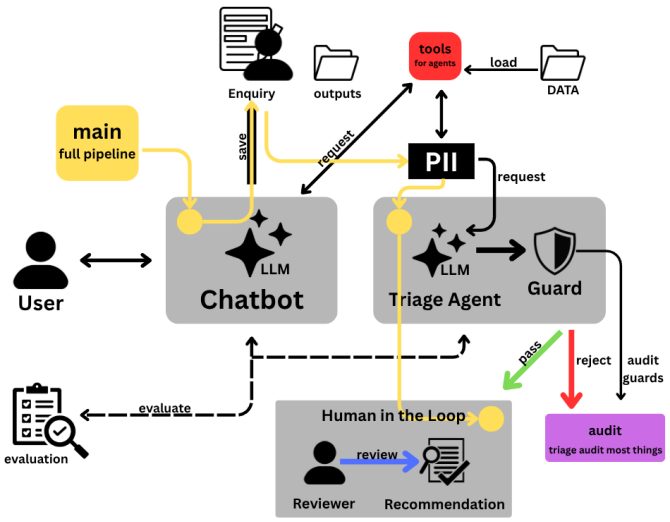


Fig. 2. System architecture. The yellow path is the pipeline run by `main.py`. All triage events (model calls, tool calls and guard results) are logged to `audit/`.

B. Shared Read-Only Tools

Both agents gather information through tool calls to a JSON store loaded from `data/`. Each agent receives only the tools it needs: the chatbot can look up and search listings, buildings and locations, while the triage agent can look up listings, buildings and locations, retrieve existing enquiries, and request duplicate candidates. Any call to a tool outside an agent's list is rejected. No tool can write data.

C. Triage Agent

Before the enquiry reaches the triage agent, it passes through a PII filter. The agent receives a redacted message and two presence flags (`has_name` and `has_contact`) instead of the contact details themselves. Tool results returned to the agent are filtered in the same way, and duplicate matching on the original contact details is performed locally. The agent returns a recommendation that must pass three guards: schema validation, a check that it refers to the correct enquiry, and an evidence check that every quote appears in the redacted message or a recorded tool result.

D. Human in the Loop

If the guards pass, the recommendation and a `pending` review are saved and the reviewer is shown the original enquiry together with the recommended label, reason and evidence. The reviewer approves or overrides the recommendation, and the decision is stored as a write-once review. Review can be done in the same session or deferred with `--queue-only`; in both cases the queue contains all pending reviews. If any guard fails, the run is rejected: the failure is logged and reported, and no review is created, so a reviewer only ever sees recommendations that passed validation. Keeping a human in the loop to review and approve model output is a well-established pattern for improving the reliability and safety of LLM-based systems [5].

E. Evaluation

The evaluation harness runs the chatbot's extraction step and the triage agent against fixed test sets, bypassing the review queue. Outputs are compared with separate answer keys that are never included in the model's input.

V. Evaluation

The two LLM stages are evaluated separately, using the same code paths as the application but bypassing the human review queue. Both use `claude-haiku-5-5` with a 2,048-token output limit, fictional data, and answer keys that are never included in the model's input. Each test set was run six times to measure variation between runs. Failed runs (outputs rejected by a guard) are counted as incorrect rather than excluded.

A. Test Sets and Metrics

Triage is tested on 100 fictional enquiries in English, French and Danish: 28 `genuine`, 24 `needs_info`, 24 `spam` and 24 `duplicate`. The metrics are label accuracy, per-class precision and recall, and, for duplicates, whether the correct original enquiry was identified.

Extraction is tested on 25 fixed chatbot transcripts, including two pairs of reworded or translated conversations. Only the final extracted fields are scored (name, email, phone, move-in date, budget, listing and declined fields), with no per-turn checks. A transcript counts as correct only if every field matches. Fields are also checked for invented values (a value given when none was stated) and omissions.

B. Results

Table II summarises the six triage runs. Mean accuracy was 96.7% (580/600), ranging from 95% to 98%. Spam was never confused with another label, and spam precision was 100% in every run.

TABLE II
TRIAGE RESULTS OVER SIX RUNS (100 ENQUIRIES EACH)

Run	Accuracy	Failed	Dup. target	Cost (USD)
1	98%	0	24/24	0.109
2	95%	3	22/24	0.105
3	98%	0	24/24	0.108
4	96%	2	23/24	0.106
5	96%	2	24/24	0.106
6	97%	1	24/24	0.110
Total	**96.7%**	**8/600**	**141/144**	**0.653**

The errors fall into two groups. First, 8 of 600 runs (1.3%) were rejected by the guards: seven failed the evidence check because a quoted tool result did not appear in the recorded output, and one failed schema validation because a quote was shorter than the minimum length. In each case no recommendation was produced, so nothing incorrect reached the review queue. Second, every run labelled one `genuine` and one `needs_info` enquiry as `duplicate`, which keeps duplicate precision at about 92%. Because this error is consistent across runs, it points to a systematic weakness in duplicate detection rather than random variation. Fig. 3 shows the confusion matrix of a representative run.

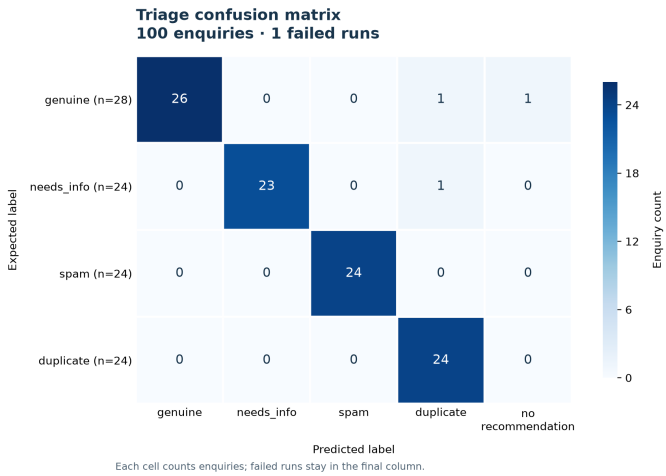


Fig. 3. Triage confusion matrix for run 6 (97% accuracy).

Extraction was correct for 147 of 150 transcripts (98%) across the six runs, with four runs scoring 25/25. No run produced an invented value or a failed extraction, and both consistency pairs matched in every run. Two of the three errors came from the same transcript (CHAT-025), where the user selects a listing by referring back to an earlier option; the model omitted the move-in date and listing in both cases. The third error was a field wrongly recorded as declined.

The estimated API cost was about $0.0011 per triaged enquiry and $0.00016 per extraction, based on recorded token usage including failed runs.

C. Limitations

These results come from synthetic data written by the author, so they show that the system behaves as designed but do not predict accuracy on real enquiries. The answer keys have not yet been independently reviewed. Six runs show the variation between runs but are not a full repeatability study. Extraction is scored on final fields only, so the quality of the chatbot's follow-up questions is not measured.

VI. CONCLUSION

This project built a pipeline that takes a rental enquiry from conversation to a human decision. A chatbot collects and extracts the enquiry, a triage agent recommends a label, and a reviewer makes the final call. The design choices aim to limit what the model can affect: it has read-only tools, it never sees contact details, and its output is validated against evidence before anyone acts on it.

The evaluation suggests that this approach works on the test data. Triage averaged 96.7% accuracy over six runs, extraction was correct in 98% of cases, and no extracted value was invented. Most importantly, every malformed or unsupported recommendation was stopped by the guards, so the remaining errors reached the reviewer as recommendations that could be overridden, not as final decisions.

The evaluation also exposed one systematic weakness: in every run, two non-duplicate enquiries were labelled as duplicates, which shows that duplicate detection needs stricter evidence. Beyond that, the main limitations are the synthetic test data, answer keys that have not been independently reviewed, regex-based redaction, and a single-reviewer design with JSON storage that would not support concurrent use.

Future work would address these in order of impact: tightening the duplicate rule, having the answer keys reviewed and testing on real (anonymised) enquiries, replacing regex redaction with a named-entity model, evaluating the chatbot's conversations turn by turn rather than only its final output, and moving to a database with multi-reviewer support.

REFERENCES

[1] Luxury Presence, "Speed to lead in real estate: Why response time wins deals," https://www.luxurypresence.com/blogs/speed-to-lead-real-estate/.
[2] Office of the Privacy Commissioner of Canada, "Pipeda legislation and related regulations," https://www.priv.gc.ca/en/privacy-topics/privacy-laws-in-canada/the-personal-information-protection-and-electronic-documents-act-pipeda, accessed: Nov. 2, 2026.
[3] Les Publications du Québec, "P-39.1 - act respecting the protection of personal information in the private sector."
[4] Anthropic, "Api overview," https://platform.claude.com/docs/en/api/overview, accessed: October. 22, 2026.
[5] H. P. Zou, W.-C. Huang, Y. Wu *et al.*, "LLM-based human-agent collaboration and interaction systems: A survey," in *Findings of the Association for Computational Linguistics: ACL 2026*. San Diego, CA, USA: Association for Computational Linguistics, Jul. 2026, pp. 36 335–36 364.